1) Develop a program to Load a dataset and select one numerical column. Compute mean, median, mode, standard deviation, variance, and range for a given numerical column in a dataset. Generate a histogram and boxplot to understand the distribution of the data. Identify any outliers in the data using IQR. Select a categorical variable from a dataset. Compute the frequency of each category and display it as a bar chart or pie chart.

Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Create the dataset based on the provided data
data = pd.DataFrame({
    'Age': [23, 23, 27, 27, 39, 41, 47, 49, 50, 52, 54, 54, 56, 57, 58, 58, 60, 61],
    '%Fat': [7.8, 9.5, 17.8, 25.9, 26.5, 27.2, 27.4, 28.8, 30.2, 31.2, 31.4, 32.9, 33.4, 34.1,
34.6, 35.7, 41.2, 42.5]
})

# Save the dataset to a CSV file
data.to_csv('Age_Fat.csv', index=False)

# Load the dataset
print("Dataset Loaded from Age_Fat.csv")
data = pd.read_csv('Age_Fat.csv')

# Preview the dataset
print("Dataset Preview:")
print(data.head())

# Numerical column: %Fat
numerical_column = '%Fat'
```

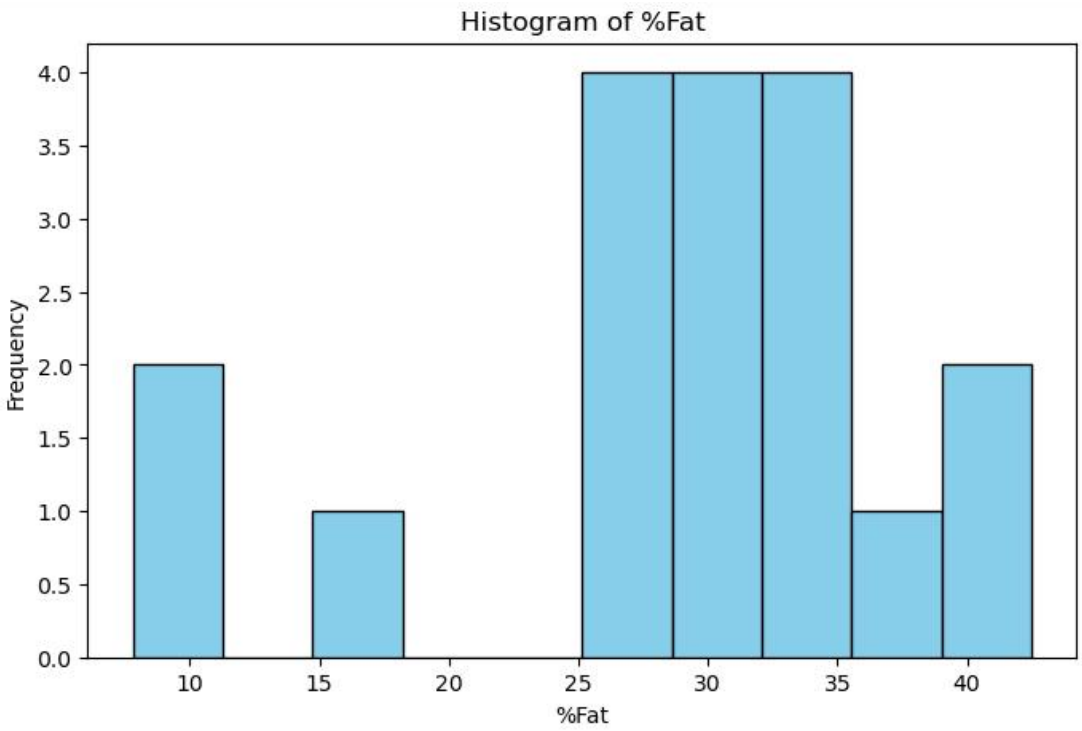
```
data_num = data[numerical_column]

# Compute statistics
mean_val = data_num.mean()
median_val = data_num.median()
mode_val = data_num.mode()[0]
std_dev = data_num.std()
variance = data_num.var()
range_val = data_num.max() - data_num.min()

# Print statistics
print("\nStatistics for Numerical Column:")
print(f"Mean: {mean_val}")
print(f"Median: {median_val}")
print(f"Mode: {mode_val}")
print(f"Standard Deviation: {std_dev}")
print(f"Variance: {variance}")
print(f"Range: {range_val}")

# Generate a histogram
plt.figure(figsize=(8, 5))
plt.hist(data_num, bins=10, color='skyblue', edgecolor='black')
plt.title(f"Histogram of {numerical_column}")
plt.xlabel(numerical_column)
plt.ylabel("Frequency")
plt.show()

# Generate a boxplot
plt.figure(figsize=(8, 5))
sns.boxplot(x=data_num, color='lightgreen')
plt.title(f"Boxplot of {numerical_column}")
plt.show()

# Identify outliers using IQR
```

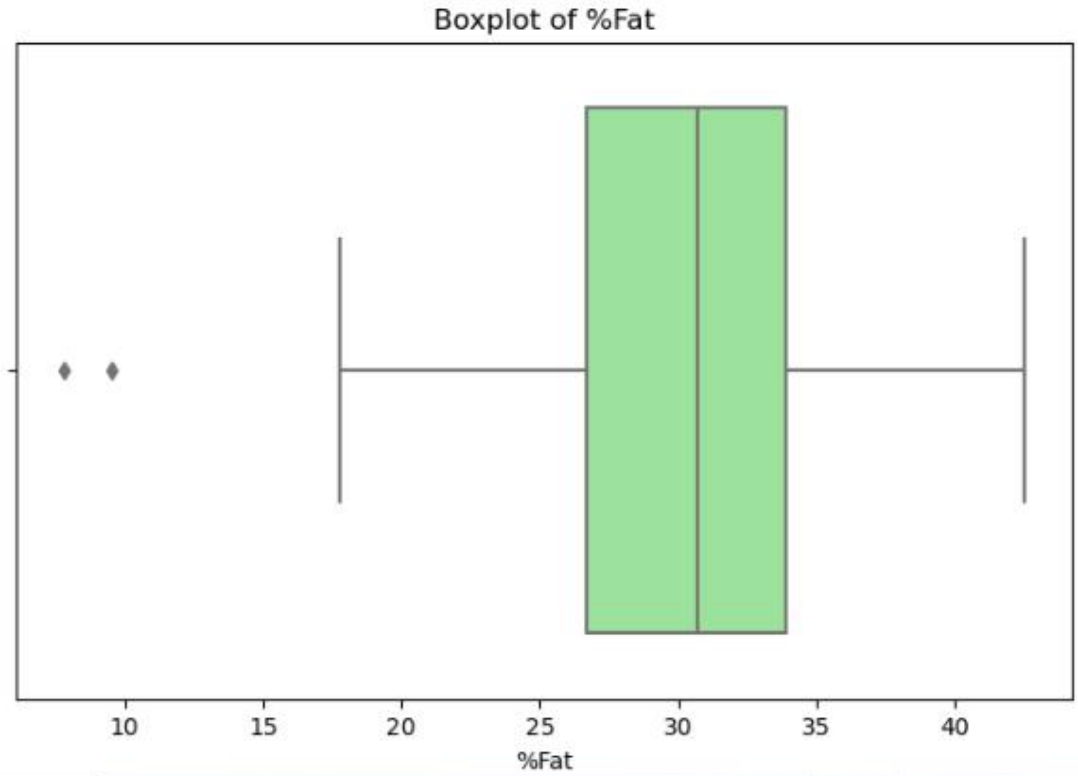
```
q1 = data_num.quantile(0.25)
q3 = data_num.quantile(0.75)
iqr = q3 - q1
lower_bound = q1 - 1.5 * iqr
upper_bound = q3 + 1.5 * iqr
outliers = data_num[(data_num < lower_bound) | (data_num > upper_bound)]

print("\nOutliers:")
print(outliers)

# Add a categorical column (Age Group) for demonstration purposes
def age_group(age):
    if age < 30:
        return 'Young'
    elif 30 <= age <= 50:
        return 'Middle-aged'
    else:
        return 'Older'

data['Age Group'] = data['Age'].apply(age_group)

# Categorical column: Age Group
categorical_column = 'Age Group'
data_cat = data[categorical_column]

# Compute frequency of each category
category_counts = data_cat.value_counts()

print("\nCategory Frequencies:")
print(category_counts)

# Bar chart for the categorical column
plt.figure(figsize=(8, 5))
category_counts.plot(kind='bar', color='coral', edgecolor='black')
```

```
plt.title(f"Bar Chart of {categorical_column}")
plt.xlabel(categorical_column)
plt.ylabel("Frequency")
plt.show()

# Pie chart for the categorical column
plt.figure(figsize=(8, 5))
category_counts.plot(kind='pie', autopct='%1.1f%%', startangle=90,
colors=sns.color_palette('pastel'))
plt.title(f"Pie Chart of {categorical_column}")
plt.ylabel("")  # Remove y-axis label
plt.show()
```

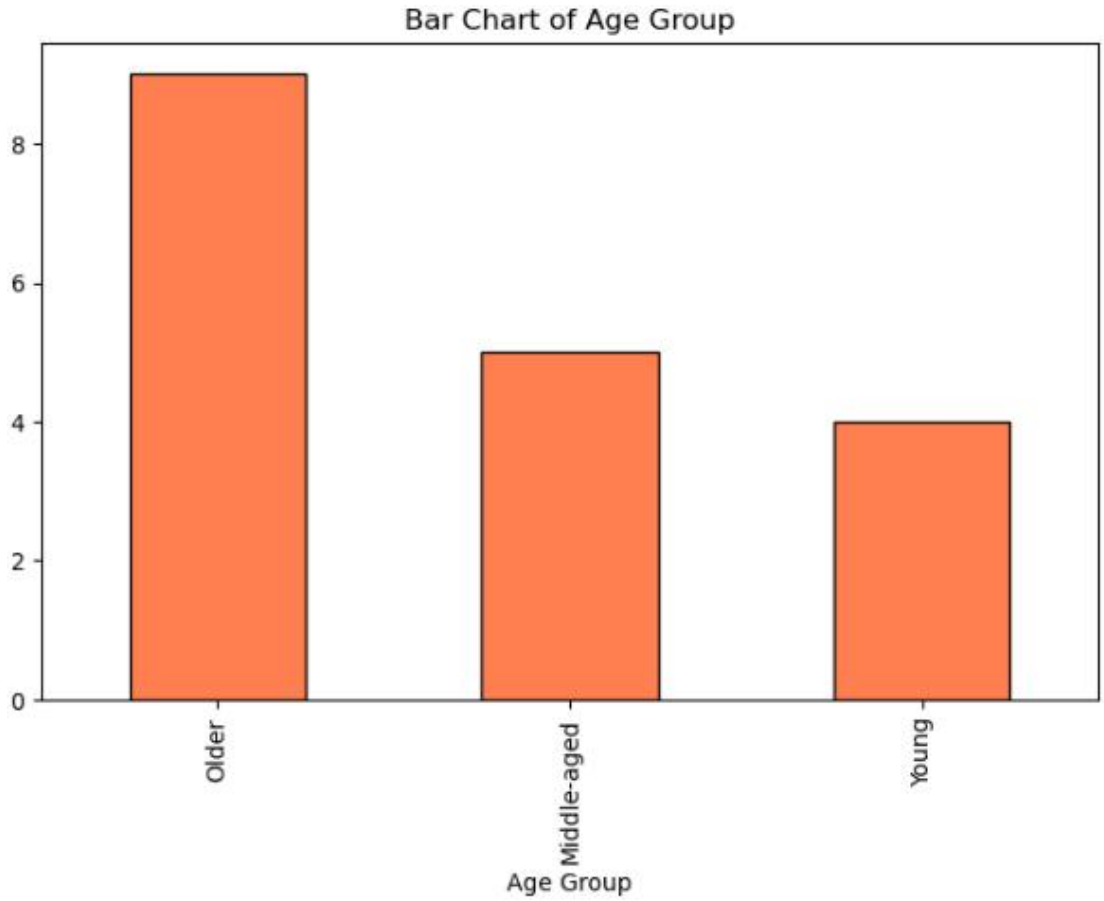
Output:

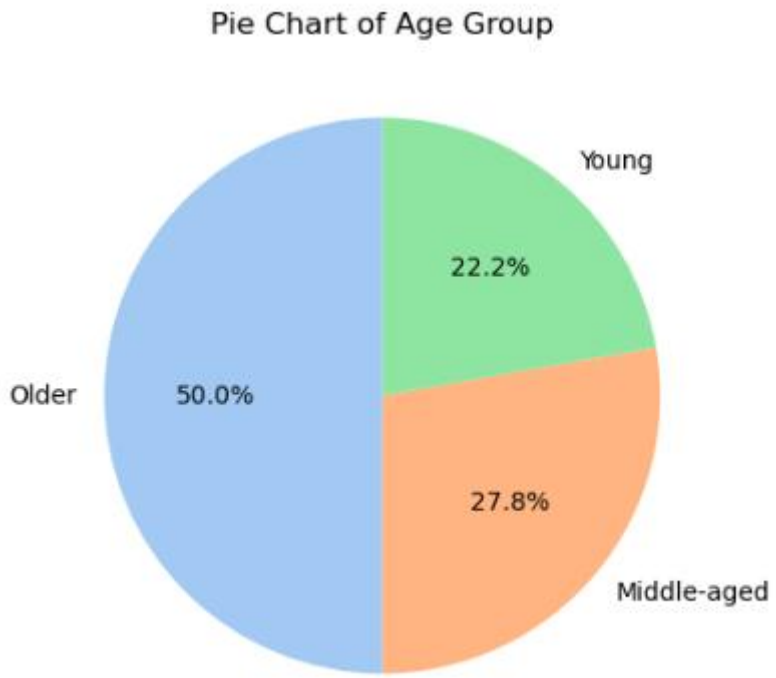
```
Dataset Loaded from Age_Fat.csv
Dataset Preview:
   Age  %Fat
0   23   7.8
1   23   9.5
2   27  17.8
3   27  25.9
4   39  26.5

Statistics for Numerical Column:
Mean: 28.783333333333328
Median: 30.7
Mode: 7.8
Standard Deviation: 9.2543948224296
Variance: 85.64382352941178
Range: 34.7
```


Histogram of %Fat
Frequency
4.0
3.5
3.0
2.5
2.0
1.5
1.0
0.5
0.0
10
15
20
25
30
35
40
%Fat


Boxplot of %Fat
10
15
20
25
30
35
40
%Fat

```
Outliers:
0    7.8
1    9.5
Name: %Fat, dtype: float64

Category Frequencies:
Age Group
Older          9
Middle-aged    5
Young          4
Name: count, dtype: int64
```





jupyter Age_Fat.csv

File Edit View Language

```
Age,%Fat
23,7.8
23,9.5
27,17.8
27,25.9
39,26.5
41,27.2
47,27.4
49,28.8
50,30.2
52,31.2
54,31.4
54,32.9
56,33.4
57,34.1
58,34.6
58,35.7
60,41.2
61,42.5
```

2) Develop a program to load the Iris dataset. Implement the k-Nearest Neighbors (k-NN) algorithm for classifying flowers based on their features. Split the dataset into training and testing sets and evaluate the model using metrics like accuracy and F1-score. Test it for different values of *k* (e.g., k=1,3,5) and evaluate the accuracy. Extend the k-NN algorithm to assign weights based on the

distance of neighbors (e.g., *weight* = $1/d^2$). Compare the performance of weighted k-NN and regular k-NN on a synthetic or real-world dataset.

Code:

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, f1_score, confusion_matrix
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler, LabelEncoder

# Load the Iris dataset from a CSV file
file_path = 'IRIS.csv'
df = pd.read_csv(file_path)

# Separate features (X) and labels (y)
X = df.iloc[:, :-1].values
y = df.iloc[:, -1].values

# Convert species labels to numerical values
label_encoder = LabelEncoder()
y = label_encoder.fit_transform(y)

# Split the dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
# Standardize features
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# --------- Custom 1/d^2 Weight Function ----------
def inverse_square_weight(distances):
    # Adding small epsilon to avoid division by zero
```

```
    return 1 / (distances**2 + 1e-5)
# ------------------------------------------------
# Function to evaluate k-NN
def evaluate_knn(X_train, X_test, y_train, y_test, k_values, weighted=False):
    results = {}

    for k in k_values:

        if weighted:
            knn = KNeighborsClassifier(
                n_neighbors=k,
                weights=inverse_square_weight   # <-- 1/d^2 applied here
            )
        else:
            knn = KNeighborsClassifier(n_neighbors=k)

        # Train model
        knn.fit(X_train, y_train)

        # Predict
        y_pred = knn.predict(X_test)

        # Evaluate
        accuracy = accuracy_score(y_test, y_pred)
        f1 = f1_score(y_test, y_pred, average='weighted')
        conf_matrix = confusion_matrix(y_test, y_pred)

        results[k] = {
            'accuracy': accuracy,
            'f1_score': f1,
            'conf_matrix': conf_matrix
        }

    return results
```

```
# Test different k values
k_values = [1, 3, 5]

# Regular k-NN
regular_knn_results = evaluate_knn(
    X_train, X_test, y_train, y_test, k_values, weighted=False
)

print("Regular k-NN Results:")
for k, metrics in regular_knn_results.items():
    print(f"\nk={k}: Accuracy={metrics['accuracy']:.4f}, "
          f"F1-Score={metrics['f1_score']:.4f}")
    print("Confusion Matrix:")
    print(metrics['conf_matrix'])

# Weighted k-NN (1/d^2)
weighted_knn_results = evaluate_knn(
    X_train, X_test, y_train, y_test, k_values, weighted=True
)

print("\nWeighted k-NN Results (1/d^2):")
for k, metrics in weighted_knn_results.items():
    print(f"\nk={k}: Accuracy={metrics['accuracy']:.4f}, "
          f"F1-Score={metrics['f1_score']:.4f}")
    print("Confusion Matrix:")
    print(metrics['conf_matrix'])
```

Output:

```
Regular k-NN Results:
k=1: Accuracy=0.9778, F1-Score=0.9777
Confusion Matrix:
[[19  0  0]
 [ 0 12  1]
 [ 0  0 13]]
```

k=3: Accuracy=1.0000, F1-Score=1.0000

Confusion Matrix:

```
[[19  0  0]
 [ 0 13  0]
 [ 0  0 13]]
```

k=5: Accuracy=1.0000, F1-Score=1.0000

Confusion Matrix:

```
[[19  0  0]
 [ 0 13  0]
 [ 0  0 13]]
```

Weighted k-NN Results (1/d^2):

k=1: Accuracy=0.9778, F1-Score=0.9777

Confusion Matrix:

```
[[19  0  0]
 [ 0 12  1]
 [ 0  0 13]]
```

k=3: Accuracy=1.0000, F1-Score=1.0000

Confusion Matrix:

```
[[19  0  0]
 [ 0 13  0]
 [ 0  0 13]]
```

k=5: Accuracy=1.0000, F1-Score=1.0000

Confusion Matrix:

```
[[19  0  0]
 [ 0 13  0]
 [ 0  0 13]]
```

3) Develop a program to Load a dataset with at least two numerical columns (e.g., Iris, Titanic). Plot a scatter plot of two variables and calculate their Pearson cor

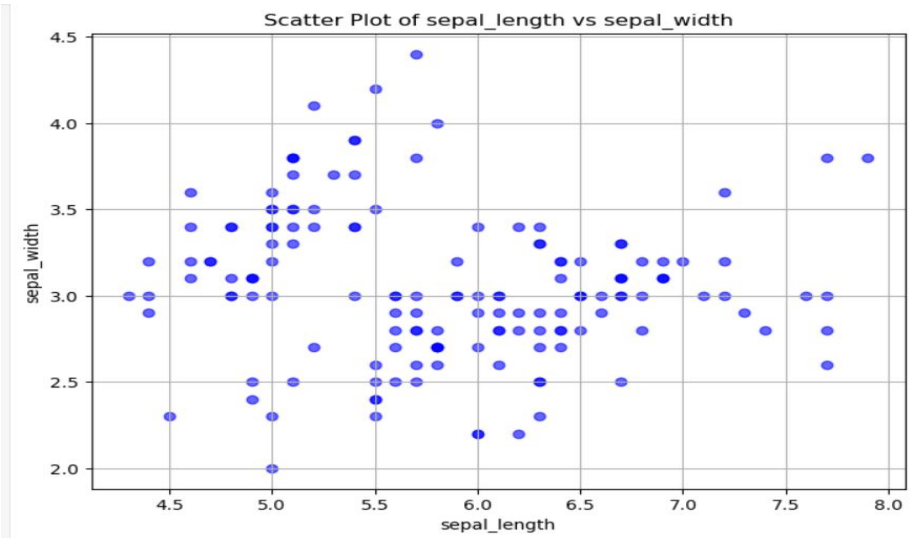
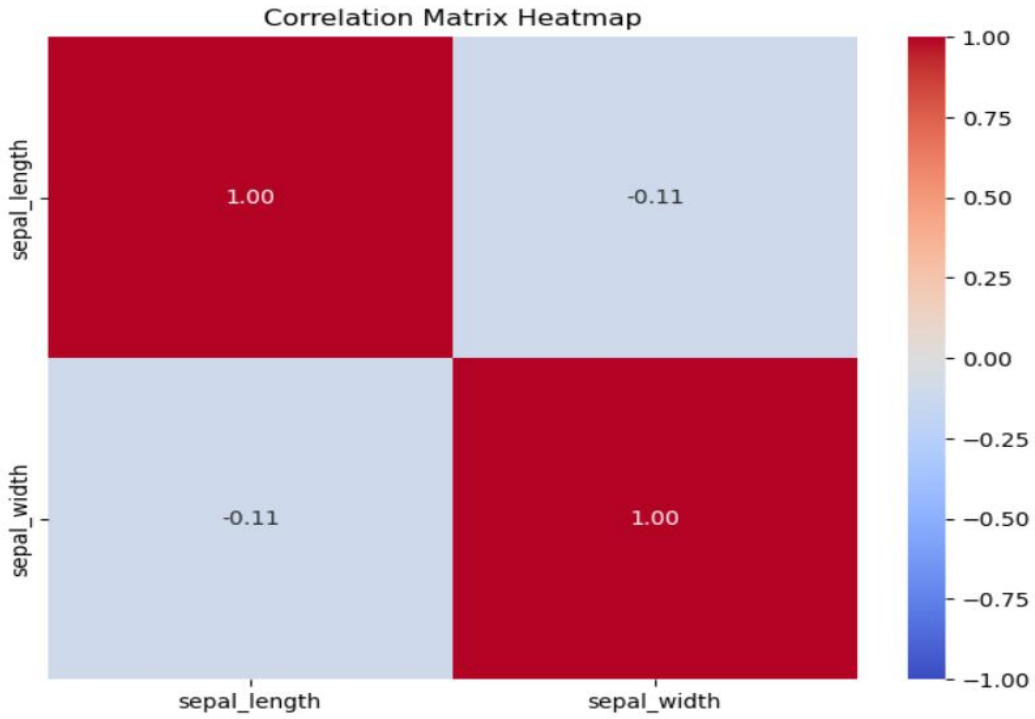
relation coefficient. Write a program to compute the covariance and correlation matrix for a dataset. Visualize the correlation matrix using a heatmap to know w hich variables have strong positive/negative correlations.

Code:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Load dataset (update 'your_dataset.csv' with your actual file path)
#file_path = "Sample_Marks.csv"  # Replace with your dataset file path
file_path = "IRIS.csv"
df = pd.read_csv(file_path)

# Extract two columns (update 'Column1' and 'Column2' with actual column names)
#column1 = "Test Score"  # Replace with the first column name
#column2 = "Video Game"  # Replace with the second column name
column1 = "sepal_length"
column2 = "sepal_width"
selected_data = df[[column1, column2]]

# Scatter plot of the selected columns
plt.figure(figsize=(8, 6))
plt.scatter(selected_data[column1], selected_data[column2], color="blue", alpha=0.6)
#plt.scatter(selected_data[column1], selected_data[column2], color="blue")
plt.title(f"Scatter Plot of {column1} vs {column2}")
plt.xlabel(column1)
plt.ylabel(column2)
plt.grid(True)
plt.show()

# Calculate Pearson correlation coefficient
correlation_coefficient = np.corrcoef(selected_data[column1], selected_data[column
2])[0, 1]
```

```
print(f"Pearson Correlation Coefficient ({column1} vs {column2}): {correlation_coef
ficient}")

# Compute covariance matrix
cov_matrix = np.cov(selected_data[column1], selected_data[column2])
print("\nCovariance Matrix:")
print(cov_matrix)

# Compute correlation matrix
correlation_matrix = selected_data.corr()
print("\nCorrelation Matrix:")
print(correlation_matrix)

# Visualize the correlation matrix using a heatmap
plt.figure(figsize=(8, 6))
#sns.heatmap(correlation_matrix, annot=True, cmap="coolwarm", fmt=".2f", cbar=Tr
ue)
#sns.heatmap(correlation_matrix, annot=True, cmap="viridis", fmt=".2f", cbar=True)
sns.heatmap(correlation_matrix,cmap="coolwarm",vmin=-1,vmax=1, annot=True, fm
t=".2f", cbar=True)
#sns.heatmap(correlation_matrix,cmap="RdBu",vmin=-1,vmax=1, annot=True)
plt.title("Correlation Matrix Heatmap")
plt.show()
```

Output:



```
Pearson Correlation Coefficient (sepal_length vs sepal_width): -0.10936924995064937

Covariance Matrix:
[[ 0.68569351 -0.03926846]
 [-0.03926846  0.18800403]]

Correlation Matrix:
              sepal_length  sepal_width
sepal_length      1.000000    -0.109369
sepal_width      -0.109369     1.000000
```